ЛАБОРАТОРНА РОБОТА № 5

«Проєктування системи класів»

Навчальна дисципліна: WEB-програмування
Варіант: 2 — «Онлайн-магазин»
Виконав: студент групи КН-310 Мельник Назар
Місто і рік: Львів — 2026

1. Мета роботи

Навчитися створювати класи JavaScript, реалізовувати наслідування та організовувати код у різних файлах за допомогою ES-модулів.

2. Короткі теоретичні відомості

Клас у JavaScript є синтаксичною надбудовою над прототипним наслідуванням. Метод `constructor` ініціалізує властивості нового екземпляра, а методи класу розміщуються в його прототипі. `extends` створює похідний клас, а `super(...)` викликає конструктор батьківського класу до використання `this`. `export` та `import` дають змогу зберігати класи в окремих файлах.

Приватні поля, геттери й сеттери описані у теоретичній частині методички, але не входять до обов'язкових вимог варіанта 2.

3. Завдання за методичкою

Для парного номера 14 виконується варіант 2 — «Онлайн-магазин»:

1. Створити `Product` із назвою, ціною й категорією.
2. Створити `User` з ім'ям і методом отримання інформації.
3. Створити `Customer`, що наслідує `User`, з переліком замовлень та методами додавання і перегляду замовлень.
4. Створити `Order` зі списком продуктів і загальною сумою.
5. Створити демонстраційні об'єкти, додати замовлення покупцю та вивести його замовлення.

4. Таблиця відповідності вимог

Вимога	Файл	Реалізація	Результат перевірки
Product: назва, ціна, категорія	`models/Product.js`	`constructor(name, price, category)`	поля ініціалізуються правильно
User: ім'я та інформація	`models/User.js`	`name`, `getInfo()`	повертає `Користувач: Олександр`
Наслідування Customer від User	`models/Customer.js`	`extends User`, `super(name)`	`instanceof User === true`; ім'я успадковано
Список замовлень	`models/Customer.js`	`orders = []`, `addOrder`, `getOrders`	порожній, один і кілька записів перевірені
Order: продукти й сума	`models/Order.js`	`products`, `reduce(...price, 0)`	2300; порожній список — 0; дробові ціни — 12.75
ES-модулі	усі JS-файли	`export` / `import`, `type="module"`	п'ять модулів завантажено з HTTP 200
Демонстрація	`index.js`	2 товари, Customer, Order, виведення	1 замовлення на 2300 грн

5. Хід виконання

Знайдено готовий проєкт із `index.html`, `index.js`, `package.json` і чотирма класами в каталозі `models`. Реалізацію повністю звірено з методичкою. Розбіжностей не виявлено, тому правильний вихідний код не переписувався.

5.1. Product і User

```javascript
export class Product {
constructor(name, price, category) {
this.name = name;
this.price = price;
```

this.category = category;
}
}

export class User {
constructor(name) {
this.name = name;
}

```
getInfo() {
    return `Користувач: ${this.name}`;
}
```

}
```

### 5.2. Customer, наслідування і super

```javascript
export class Customer extends User {
constructor(name) {
super(name);
this.orders = [];
}
```

```
addOrder(order) {
    this.orders.push(order);
}

getOrders() {
    return this.orders;
}
```

}
```

`Customer` успадковує `name` і `getInfo()` від `User`. Виклик `super(name)` ініціалізує батьківську частину об'єкта до звернення до `this.orders`.

5.3. Order і обчислення суми

`javascript export class Order { constructor(products)`

```
{ this.products = products; this.total = products.reduce((sum, product) => sum + product.price, 0); } }
```

Початкове значення `reduce` дорівнює 0, тому порожнє замовлення коректно має суму 0. Для демонстрації: `800 + 1500 = 2300 грн`.

5.4. Демонстраційні об'єкти

`index.js` створює мишу, клавіатуру, покупця Олександра та замовлення з двох товарів. `customer.addOrder(order)` додає його до списку, після чого виводяться інформація про користувача, товари, кількість замовлень і сума.

6. Фактичні перевірки

6.1. Node.js

Виконано синтаксичну перевірку п'яти JS-файлів і демонстрацію:

```
bash node --check index.js node --check models/Product.js node --check models/User.js node --check models/Customer.js node --check models/Order.js node index.js
```

Результат: `SYNTAX_CHECKS_OK=5`; демонстраційна сума — 2300 грн.

6.2. Автоматичні ООР-тести

Пройдено **15 тестів, 0 невдалих**. Перевірено конструктори й поля, `getInfo`, `Customer instanceof User`, успадкований метод, результат `super(name)`, початковий список замовлень, додавання одного та кількох замовлень, збереження списку продуктів, суму 2300, порожню суму 0 та дробову суму 12.75.

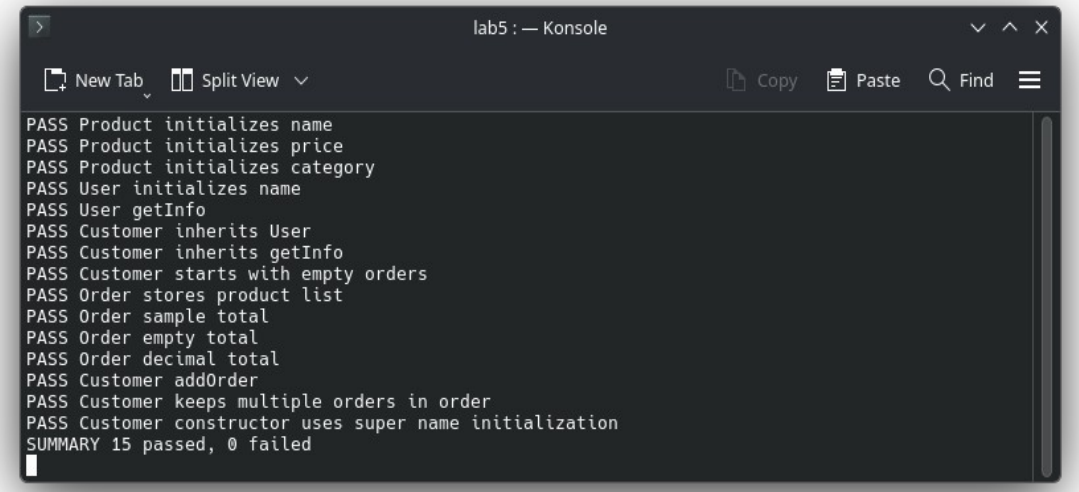


Рисунок 1 — Реальний результат 15 автоматичних ООР-тестів у Konsole.

6.3. Браузерна перевірка Playwright

Playwright відкрив сторінку через локальний HTTP-сервер і підтвердив:

- HTTP 200 для сторінки, `index.js` і чотирьох модулів класів;
- наявність `<script type="module" src="./index.js">`;
- шість точних console-повідомлень демонстрації;
- `page_errors=0` і `request_failures=0`.

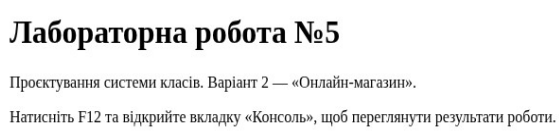


Рисунок 2 — Сторінка WEB ЛР №5, відкрита Playwright.

6.4. Результати демонстрації

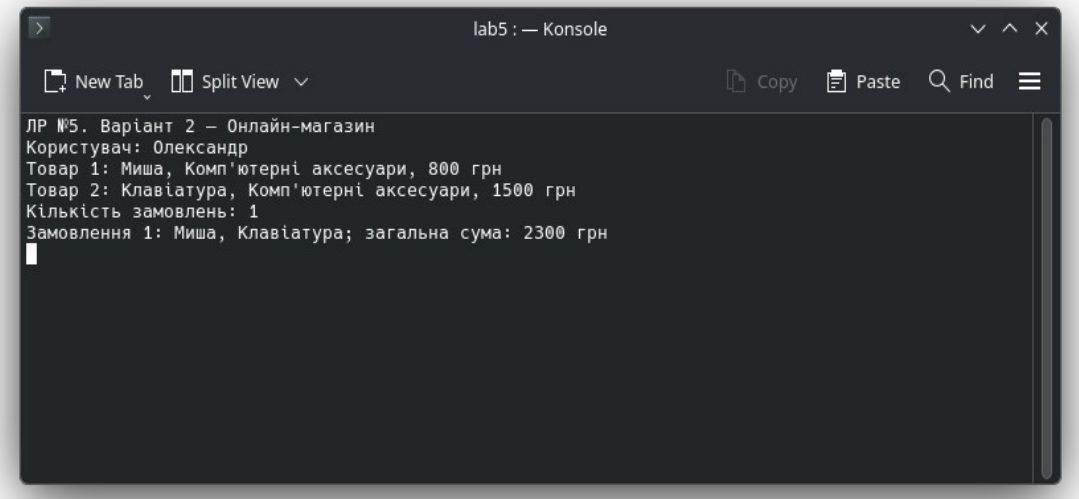


Рисунок 3 — Фактичне виконання `node index.js`*: покупець, два товари, одне замовлення і сума 2300 грн.*

Браузерні console-події з тим самим результатом збережено без імітації в `logs/04_playwright.txt`.

7. Відповіді на питання вихідного контролю

7.1. Що таке клас у JavaScript?

Клас — синтаксична конструкція для створення об'єктів зі спільними властивостями й методами, що працює поверх прототипної моделі JavaScript.

7.2. Як реалізується успадкування?

Ключовим словом `extends`, наприклад `class Customer extends User`. Похідний клас успадковує доступні методи та властивості базового класу.

7.3. Яка різниця між класами та функціями-конструкторами?

Обидва механізми використовують прототипи, але `class` має спеціальний синтаксис методів і наслідування, працює у strict mode та не викликається без `new`.

7.4. Як організувати класи в різних файлах?

Кожен клас експортують з окремого ES-модуля через `export`, а залежності підключають через `import`. `Customer.js` імпортує `User`, а `index.js` — `Product`, `Customer` і `Order`.

7.5. Як працює super у конструкторі нащадка?

`super(...)` викликає конструктор батьківського класу. У похідному конструкторі його потрібно викликати до використання `this`.

8. Обмеження

Git-гілку `lab5`, посилання в меню `labs/index.html`, merge, push і GitHub Pages не підтверджено: окремого навчального репозиторію `labs` та даних доступу до публікації немає. Git push і публікація не виконувались.

9. Висновки

У роботі реалізовано класи Product, User, Customer й Order, перевірено конструктори, методи, наслідування, super та ES-модулі. Покупцю додано замовлення з двох товарів на суму 2300 грн. Синтаксичні й алгоритмічні перевірки завершилися успішно; браузерні результати зафіксовано в журналах Codex. Вихідний код не змінювали.